

实验七：用户信息管理系统

实验七：用户信息管理系统

实验目的

1. 掌握 RESTful API 设计规范
2. 学习用户管理系统的完整实现
3. 理解数据校验和错误处理机制
4. 掌握单元测试的编写方法

实验内容

实验过程

1. 代码结构分析 本实验包含两个 Python 文件：

user_manager.py - 用户管理系统核心 - `validate_user_id()`: 校验用户 ID 合法性 - `validate_username()`: 校验用户名合法性 - `validate_age()`: 校验年龄合法性 - `validate_gender()`: 校验性别合法性 - `add_user()`: 用户新增 - `get_user_by_id()`: 单用户查询 - `get_all_users()`: 批量查询 - `update_user()`: 用户修改 - `delete_user_by_id()`: 单用户删除 - `delete_users_by_ids()`: 批量删除

test_user_manager.py - 完整测试脚本 - 用户新增功能测试（8 个用例）- 用户查询功能测试（5 个用例）- 用户修改功能测试（6 个用例）- 用户删除功能测试（7 个用例）- 联动流程集成测试（3 个流程）

2. 数据校验机制 通用校验函数

```
def validate_user_id(user_id):  
    """校验用户 ID 合法性（正整数）"""  
    if not isinstance(user_id, int) or user_id <= 0:  
        return False, "ID 不合法，需为正整数"  
    return True, ""  
  
def validate_username(username):
```

```
"""校验用户名合法性（2-10 位，字母、数字、下划线）"""
if not isinstance(username, str) or len(username) < 2 or len(username) > 10:
    return False, "用户名长度需为 2-10 位"
for char in username:
    if not (char.isalpha() or char.isdigit() or char == '_'):
        return False, "用户名仅允许包含字母、数字、下划线"
return True, ""

def validate_age(age):
    """校验年龄合法性（1-120 岁，正整数）"""
    if not isinstance(age, int) or age < 1 or age > 120:
        return False, "年龄需为 1-120 岁的正整数"
    return True, ""

def validate_gender(gender):
    """校验性别合法性（仅允许男、女、未知）"""
    valid_genders = ["男", "女", "未知"]
    if not isinstance(gender, str) or gender not in valid_genders:
        return False, "性别仅允许输入'男'、'女'、'未知'"
    return True, ""
```

3. CRUD 功能实现 用户新增

```
def add_user(user_id, username, age, gender):
    # 校验所有输入参数
    id_valid, id_msg = validate_user_id(user_id)
    if not id_valid:
        return {"status": 0, "message": id_msg, "data": {}}

    username_valid, username_msg = validate_username(username)
    if not username_valid:
        return {"status": 0, "message": username_msg, "data": {}}

    age_valid, age_msg = validate_age(age)
    if not age_valid:
        return {"status": 0, "message": age_msg, "data": {}}

    gender_valid, gender_msg = validate_gender(gender)
    if not gender_valid:
        return {"status": 0, "message": gender_msg, "data": {}}

    # 校验用户 ID 是否已存在
    if user_id in user_container:
```

```
        return {"status": 0, "message": f"用户 ID {user_id} 已存在", "data": {}}

# 新增用户到数据容器
user_info = {
    "id": user_id,
    "username": username,
    "age": age,
    "gender": gender
}
user_container[user_id] = user_info

return {"status": 1, "message": "用户新增成功", "data": user_info}
```

用户修改

```
def update_user(user_id, new_username=None, new_age=None, new_gender=None):
    # 校验用户 ID 合法性及是否存在
    id_valid, id_msg = validate_user_id(user_id)
    if not id_valid:
        return {"status": 0, "message": "修改 ID 不合法", "data": {}}

    if user_id not in user_container:
        return {"status": 0, "message": "未找到该用户", "data": {}}

    # 校验是否修改了至少一个字段
    if new_username is None and new_age is None and new_gender is None:
        return {"status": 0, "message": "未修改任何字段", "data": {}}

    # 校验修改字段的合法性
    if new_username is not None:
        username_valid, username_msg = validate_username(new_username)
        if not username_valid:
            return {"status": 0, "message": username_msg, "data": {}}

    # 更新用户信息
    user_info = user_container[user_id]
    if new_username is not None:
        user_info["username"] = new_username
    if new_age is not None:
        user_info["age"] = new_age
    if new_gender is not None:
        user_info["gender"] = new_gender
```

```
return {"status": 1, "message": "用户修改成功", "data": user_info}
```

4. 批量删除功能

```
def delete_users_by_ids(user_ids):
    # 校验参数是否为列表
    if not isinstance(user_ids, list):
        return {"status": 0, "message": "删除 ID 不合法", "data": {}}

    # 校验列表是否为空
    if len(user_ids) == 0:
        return {"status": 0, "message": "批量删除 ID 列表为空", "data": {}}

    # 执行批量删除
    success_ids = []
    not_found_ids = []

    for uid in user_ids:
        if uid in user_container:
            del user_container[uid]
            success_ids.append(uid)
        else:
            not_found_ids.append(uid)

    # 返回结果
    if len(not_found_ids) == 0:
        return {"status": 1, "message": "批量删除成功", "data": {"success_ids": success_ids, "not_found_ids": not_found_ids}}
    else:
        return {"status": 0, "message": "批量删除部分失败", "data": {"success_ids": success_ids, "not_found_ids": not_found_ids}}
```

实验结果

测试用例覆盖

模块	测试用例	通过情况
用户新增	8 个	正确
用户查询	5 个	正确
用户修改	6 个	正确
用户删除	7 个	正确
集成测试	3 个流程	正确

测试输出示例

=====

用户信息管理系统 - 完整测试

=====

【4.1 用户新增功能测试】

- T001 - 正常新增合法用户：PASS
- T002 - 重复 ID 新增：PASS
- T003 - ID 为 0 非法输入：PASS
- T004 - 用户名长度不足 2 位：PASS
- T005 - 用户名含特殊符号：PASS
- T006 - 年龄超出上限 120：PASS
- T007 - 年龄小于 1：PASS
- T008 - 性别输入非法值：PASS

【4.2 用户查询功能测试】

- T101 - 查询已存在用户 ID：PASS
- T102 - 查询不存在用户 ID：PASS
- T103 - 查询非法负数 ID：PASS
- T104 - 无任何用户：PASS
- T105 - 存在多条用户数据：PASS

【4.3 用户修改功能测试】

- T201 - 正常修改用户名：PASS
- T202 - 正常修改年龄：PASS
- T203 - 正常修改性别：PASS
- T204 - 不传入任何修改字段：PASS
- T205 - 修改不存在用户 ID：PASS
- T206 - 修改字段非法：PASS

【4.4 用户删除功能测试】

- T301 - 删除存在用户：PASS
- T302 - 删除不存在用户：PASS
- T303 - 删除非法 ID：PASS
- T304 - 全部 ID 均存在：PASS
- T305 - 部分 ID 存在部分不存在：PASS
- T306 - 传入空 ID 列表：PASS
- T307 - 列表内含非法 ID：PASS

【5. 联动流程集成测试】

- 流程 1 - 完整 CRUD 流程：PASS
- 流程 2 - 批量操作流程：PASS
- 流程 3 - 异常操作稳定性：PASS

=====

测试汇总：29/29 通过
[SUCCESS] 所有测试用例通过！

=====

实验总结

实验成果

1. 实现了完整的用户信息管理系统（CRUD）
2. 实现了完善的数据校验机制
3. 实现了批量操作功能
4. 编写了全面的测试用例（29 个）

技术要点

1. **数据校验**: 多层校验确保数据完整性和合法性
2. **统一返回格式**: 所有接口返回标准 JSON 格式
3. **批量操作**: 支持批量删除，返回详细结果
4. **错误处理**: 完善的错误处理和错误信息返回

改进建议

1. 可添加用户权限管理
2. 可实现数据持久化存储
3. 可添加分页查询功能
4. 可实现用户数据导出功能